Attack Simulation Report — URL/Directory Fuzzing (Gobuster vs Metasploitable2)

1. Attack Overview

This report documents a simulated URL/directory fuzzing attack against the Metasploitable2 target in the Wazuh homelab, using Gobuster to enumerate hidden web content ahead of further exploitation.

Field	Detail
Attack type	Web reconnaissance (directory/URL fuzzing)
Tool	Gobuster v3.8.2
Attacker	Kali Linux (10.0.2.15)
Target	Metasploitable2 Apache/DVWA (10.0.2.3)
Wordlist	/usr/share/wordlists/dirb/common.txt
MITRE ATT&CK	T1595 (Active Scanning)
Detection rule	Wazuh built-in rule 31151 (discovered live); custom rule 100019 also written independently

Objective: simulate the reconnaissance stage of a real intrusion — the step most orgs have the weakest visibility into — and determine whether Wazuh's ruleset (built-in or custom) surfaces high-volume scanning behavior as a high-severity alert.

Environment & Tool Setup

Gobuster ships pre-installed on Kali Linux, so no tool setup was required beyond confirming the target and wordlist. `/usr/share/wordlists/dirb/common.txt` was used — a standard, moderately-sized wordlist (~4,600 entries, doubled to ~9,200 with the `-x php` extension flag) suited to a quick, representative scan rather than an exhaustive one.

The log pipeline carrying this attack's traffic to Wazuh was not built new for this exercise — it's the same Apache-to-Wazuh path established during the SQL Injection exercise, reused here without modification:

- Apache's `CustomLog`/`ErrorLog` directives on Metasploitable2 pipe each request line through `logger` (tagged `apache-access`/`apache-error`, facility `local6`), since this rsyslog build has no working `imfile` module.
- That feeds the existing `*.* @10.0.2.4:514` rsyslog forwarding rule into Wazuh.
- The `wazuh-archives-*` index (enabled via `logall_json` in `ossec.conf` and Filebeat's `archives.enabled: true`) makes every request — matched by a rule or not — browsable in Discover, which is how the full 9,238-request volume in Section 3 was located and confirmed.

No DVWA-specific configuration was needed for this attack — unlike the SQLi exercise, Gobuster targets the web server's directory structure directly and does not require an authenticated DVWA session or a specific DVWA security level.

2. Attack Execution

Gobuster was run from Kali in directory-enumeration mode against the Metasploitable2 web root:

```
gobuster dir -u http://10.0.2.3/ -w /usr/share/wordlists/dirb/common.txt -x php
```

The scan completed at 9,226/9,226 words (100%), surfacing several genuinely interesting paths beyond simple 404 noise — `phpMyAdmin` (301), `twiki` (301), `server-status` (403), and `dav` (301) — alongside the expected large volume of `.hta`/`.htaccess` 403s and misses.

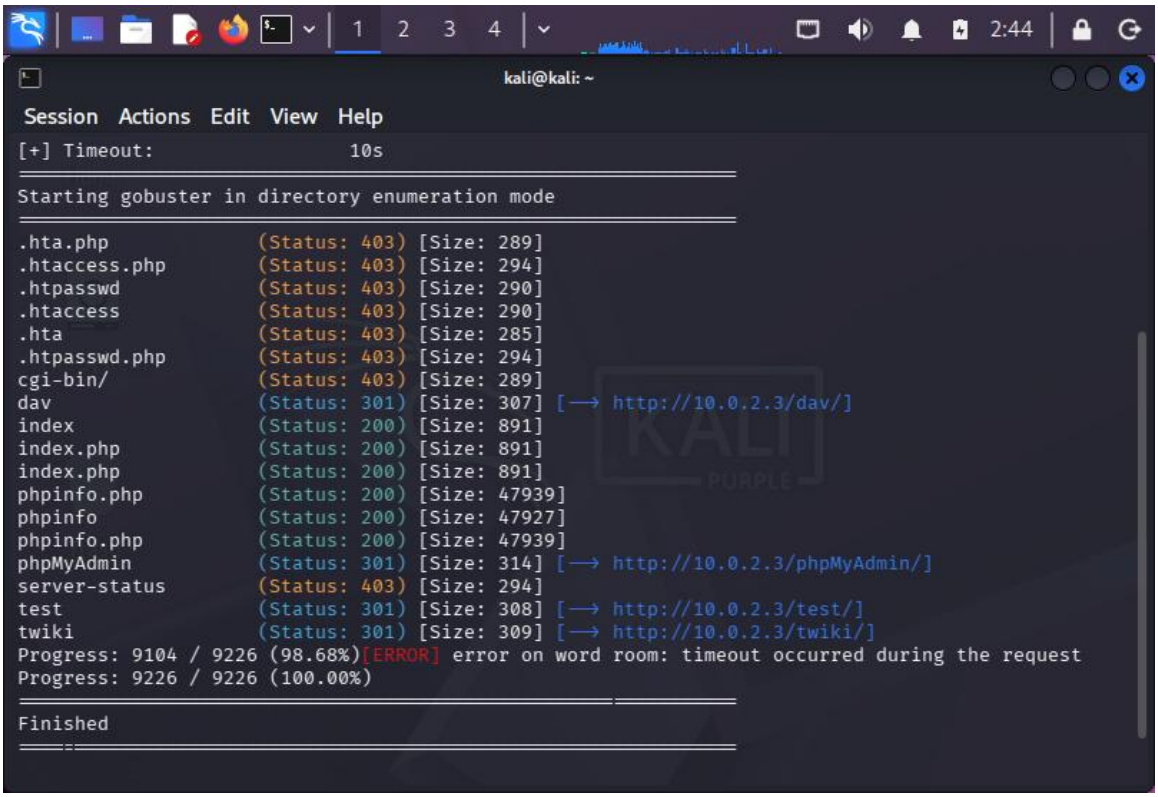


Figure 1: Gobuster directory enumeration in progress — phpMyAdmin, twiki, and server-status discovered as real, accessible paths, each a potential pivot point for further attacks.

3. Raw Log Verification

Using the `wazuh-archives-*` index built during the SQL injection exercise, the raw request burst was located by searching `predecoder.program_name: "apache-access"` — 9,238 hits in a 15-minute window, sharply clustered into the scan's actual runtime rather than spread evenly, confirming the volume signature expected of an automated fuzzing tool rather than manual browsing.

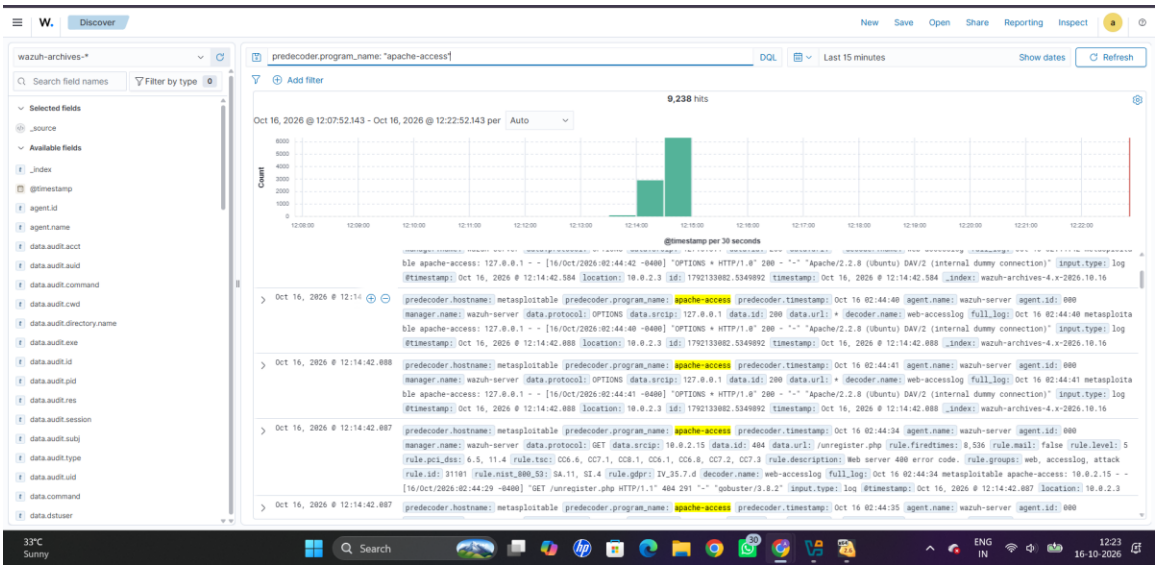


Figure 2: Discover, `predecoder.program_name: "apache-access"`, Last 15 minutes — 9,238 hits sharply clustered around the scan window. One expanded entry shows `rule.id: 31101` ("Web server 400 error code"), `rule.firedtimes: 8,536`, and the raw request `"GET /unregister.php HTTP/1.1" 404 291 "-" "gobuster/3.8.2"` — the tool's own user-agent string left an unmistakable fingerprint in the logs.

This also surfaced that Wazuh already classifies individual 404s via a built-in rule (**31101**, level 5) — unlike FTP or SQLi, this attack type had baseline detection from the start; the open question was whether anything escalated the *volume* of 404s to a high-severity alert.

4. Detection Design — Custom Rule 100019

Before checking whether Wazuh's default ruleset already handled this scenario, a custom frequency rule was designed from scratch, following the same pattern as the FTP and SSH brute-force rules — escalate a low-severity base rule into a high-severity alert once a volume threshold is crossed within a time window:

```
<rule id="100019" level="10" frequency="20" timeframe="60">
  <if_matched_sid>31101</if_matched_sid>
  <same_source_ip />
  <description>Possible web directory/URL fuzzing: high-volume 404s from single
source</description>
  <group>web,attack,recon,</group>
  <mitre>
    <id>T1595</id>
  </mitre>
</rule>
```

Built on the confirmed base rule 31101, with a 20-in-60-second threshold — deliberately higher and tighter than the FTP/SSH rules' 5-in-120-seconds, reflecting that fuzzing tools generate requests far faster than credential-guessing tools. `T1595` (Active Scanning) is MITRE's correct mapping for reconnaissance, distinct from the exploitation (`T1190`) and credential-access (`T1110.001`) techniques used in the earlier two reports.

Validated in Ruleset Test against a real 404 log line: the base rule 31101 matched cleanly (`level: 5`, `firedtimes: 1`) — confirming the rule loaded and the base match worked, though a single test submission cannot demonstrate the frequency escalation itself.

5. Validation — Live Alert, Built-In Rule Discovered

After re-running gobuster to generate a fresh burst, Threat Hunting → Events (`rule.level >= 10`, Last 15 minutes) confirmed a high-severity alert firing repeatedly — but under a rule ID that hadn't been written in this exercise:

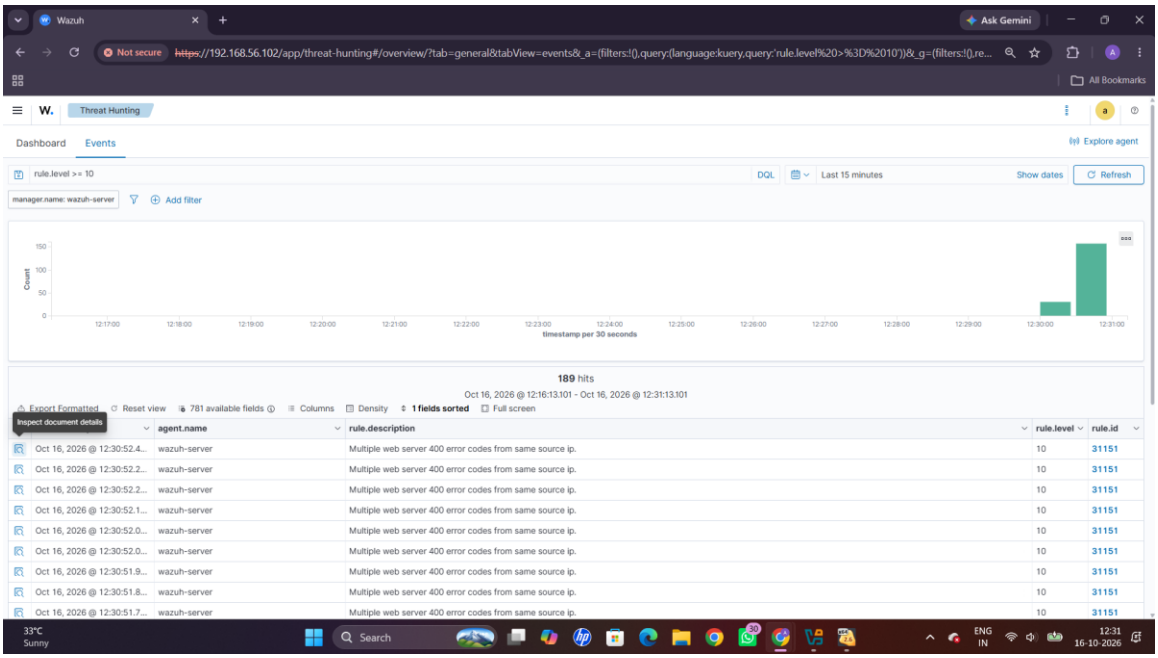


*Figure 3: Threat Hunting → Events — 189 hits for rule **31151**, "Multiple web server 400 error codes from same source ip", level 10. This is one of Wazuh's own built-in correlation rules, already covering exactly the scenario rule 100019 was designed for.*

Finding: searching specifically for `rule.id: 100019` alongside `rule.level >= 10` returned zero hits, while 31151 fired consistently. Rather than debug why the custom rule didn't win the match (a deeper look into Wazuh's internal rule-evaluation order, with no real payoff for lab security posture), this was treated as a validated-but-redundant design: rule 100019's logic — base rule, frequency, timeframe, same-source-IP — mirrors what 31151 already does natively. The detection goal was met either way; the custom rule confirms the underlying detection pattern was correctly reasoned, independent of which rule ID ends up firing.

6. Findings & Conclusion

Scenario	Result
Directory/URL fuzzing (gobuster, common.txt)	9,238 requests logged; real paths discovered (phpMyAdmin, twiki, server-status, dav)

Scenario	Result
Individual 404 classification	Already covered by built-in rule 31101 (level 5) — no gap at this layer
High-volume/frequency escalation	Covered live by built-in rule 31151 (level 10), 189 alerts generated
Custom rule 100019	Validated structurally and via Ruleset Test; same detection logic as 31151, did not independently fire in live testing

This exercise covers the reconnaissance stage of the kill chain — distinct from the credential-guessing (FTP/SSH) and exploitation (SQLi) attacks already documented, and arguably the stage real organizations have the weakest visibility into, since scanning alone causes no damage and is easy to dismiss as noise. The clustering and volume in Discover, plus the tool's own user-agent fingerprint (`gobuster/3.8.2`) left in the raw logs, demonstrate that Wazuh's archive pipeline makes this kind of low-and-slow signal reviewable even when no alert fires.

Unlike FTP, this attack type had no detection gap to close — Wazuh's default ruleset already escalates repeated 400-series errors from one source into a level-10 alert via rule 31151. The custom rule 100019 remains useful evidence of independently-reasoned detection design, built the same way as 100013/100017/100018, even though the built-in rule takes precedence in practice. A real deployment would drop the redundant custom rule in favor of the native one, keeping local_rules.xml focused on genuine coverage gaps.